



1.3 Streaming

流式输出

从"打字机效果"到可观察、可取消、可恢复的 Agent 事件流。



CLIENT · Web / CLI / IDE

subscribe · render · cancel

EVENT STORE · facts only

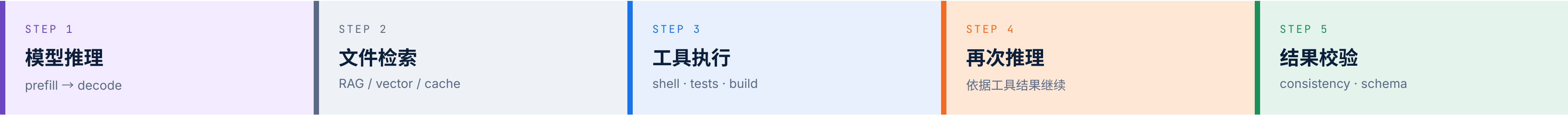
persist · replay · dedupe

↑ Harness Event Stream ↑

PATH LLM Engineering → Agent Loop → Harness | DURATION 90 min | LEVEL Intermediate | DELIVERABLE Streaming Agent Gateway

Agent 运行几十秒，用户不能只看到一个转圈图标

一次 Agent Run 可能经历 5 段状态，全部用一行 JSON 一次返回 = 完全不可观察。



如果只在最后返回 JSON，用户无法判断系统是：

- × 正在生成，还是正在排队？
- × 正在调用工具，还是已经卡死？
- × 仍在执行，还是早已失败？
- × 方向正确，还是应该立即停止？

对比 · 黑盒 vs 事件流

× 黑盒等待 30 s

0 个事件

转圈图标 · 进度条停滞 · 用户只能刷新或关闭

✓ 持续事件流

~180 事件

text.delta · tool.started · run.completed

Streaming 是 Agent Runtime 到交互层的实时事件通道。

完成本节后，你应当具备 6 项能力

一条完整工程链路 · 为什么流 → 流怎样表达 → 服务端怎样生产 → 客户端怎样消费 → 怎样停止 → 故障后怎样恢复

01 · CONCEPT

TTFT

区分三种延迟

TTFT、生成耗时与端到端延迟的工程含义。

02 · PROTOCOL

3 层

HTTP / SSE / Harness

理解 Chunk、SSE 与 Harness Event 的边界。

03 · BUILD

2.4 s

低延迟转发

DeepSeek V4 Flash + FastAPI 的端到端实现。

04 · CONSUME

3 端

Web / CLI / IDE

正确消费、解析和渲染同一事件流。

05 · CONTROL

STOP

贯穿四级取消

Client → Loop → Model → Tool 端到端信号。

06 · RESUME

3 种

恢复策略

有限重试 · 事件重放 · Checkpoint。

对应工程链路 · 一个 Harness 必须同时具备

WHY

为什么要流

EXPRESS

流怎样表达

PRODUCE

服务端怎样生产

CONSUME

客户端怎样消费

STOP

怎样停止

RECOVER

怎样恢复

6 项目标 = 6 张能力卡片。缺少任何一张，工程链路就无法支撑一个生产级 Agent。

stream=True 不会让模型推理得更快



假设一次回答

2.4 s · TTFT

首个 Token

prefill → 调度 → decode 启动

9.6 s · GENERATE

逐 Token 生成

decoding 受 GPU 与输出长度约束

12.0 s · E2E

任务完成

两者结束时间相同，用户感知不同

Streaming 改善感知延迟，并提供提前取消的机会；
它不自动减少推理时间。

把"首个结果出现"和"任务最终完成"分开测量，是 Streaming 工程的起点。

一个 latency_ms 无法解释流式体验

Metric	Definition	Main Drivers
TTFT Time to First Token	请求开始 → 首个非空文本 delta 到达客户端	· 排队 · 输入长度 · Prefill · 调度
GEN Generation Time	首 Token → 模型生成结束（含最后 usage chunk）	· 输出长度 · Decode 速度 · 模型强度
E2E End-to-End Latency	用户操作 → Agent 最终完成（含工具、Loop、Store、客户端渲染）	· 网络 · 模型 · Loop · 工具 · 存储 · 渲染

⚠ ENGINEERING CAUTION

首个 SDK chunk 不一定包含文本；TTFT 应记录首个非空 **text.delta**。



把首事件当作 TTFT，会让指标看起来更漂亮，却不代表用户真正看到了内容。

聊天流式输出文本；Agent 流式输出运行状态

一条 Harness 事件时间线

run_123 · 8 个事件 · 3 类颜色



核心问题 · 工具执行期间没有文本，旧 Streaming 制造新的"假卡顿"

✗ 只传文本的事件流

用户在 **tool.started** → **tool.completed** 之间什么也看不到。

"它是不是挂了？" 是一个不可避免的疑问。

✓ 语义事件流 · 三类颜色

生产级 Agent 应传输**语义事件**，而不是只传输字符。

■ model · ■ tool · ■ control

文本 delta 只是数据面；可靠运行依赖控制面

三类事件 · 解决的问题 · 工程含义

DATA · CONTENT

展示模型输出

只在 Last-Mile UX 层呈现，一旦 Loop 失败，这部分会消失或重复。

text.delta

```
{ "delta": "正在分析 ..." }
```

message.completed

完整 assistant message 的最终结果

没有这部分 → 用户只看到"另一端的 Chat Demo"。

PROCESS · PROGRESS

暴露 Loop 进度

把"模型让我等了多少秒"翻译成可见事件，告诉用户与系统到底走到哪一步。

tool.started

```
{ "name": "shell", "call_id": "c_017" }
```

tool.completed

```
{ "call_id": "c_017", "ok": true }
```

run.retrying

```
{ "attempt": 2, "reason": "network" }
```

没有这部分 → 系统是一头看不见在做什么的黑盒。

CONTROL · TERMINAL

表达唯一终态

一个 Run 只有一个终态；取消和完成发生竞态时，必须靠这条事件做最终裁判。

run.completed

状态 = ok

run.failed

```
{ "code": "TOOL_TIMEOUT" }
```

run.cancelled

```
{ "cancelled_at": "tool:call:c_017" }
```

没有这部分 → 完成与取消的竞态会留下两条"终态"。

状态、取消、错误和恢复，才是 Agent Streaming 的控制面。

只实现 text.delta + tool.completed → 聊天 Demo；补齐 run.cancelled / failed / completed → 才成为 Harness。

Streaming 能改善反馈，但不能替代系统设计

✓ CAN SOLVE

能解决 5 件事

✓ 01
尽早展示已生成内容
TTFT 决定用户能否在 2-3 s 内看到动作。

✓ 02
展示长任务执行进度
过程事件替换转圈图标。

✓ 03
允许用户及时纠偏和取消
控制面事件是纠偏的工程基础。

✓ 04
为 Web / CLI / IDE 提供同一事件流
协议一致 → 客户端差异化只在渲染层。

✓ 05
增量记录一次 Run 的事实轨迹
Event Store 让轨迹可审计、可重放。

✗ CANNOT SOLVE

不能解决 5 件事

✗ 01
模型推理和工具本身过慢
GPU 与 IO 限制不会被 SSE 缓解。

✗ 02
Prompt 过长导致 Prefill 昂贵
上下文管理 ≠ 流式协议。

✗ 03
事件未持久化导致进程崩溃丢失
没有 Event Store 就谈不上恢复。

✗ 04
客户端断线后任务继续消耗资源
没有取消就永远在扣预算。

✗ 05
Context Window 的输入输出上限
不是传输问题，是模型能力问题。

三个不同的问题：Streaming = 链路传输 · Checkpoint = 任务状态 · Context Window = 模型能力。任何一项都不能替代另外两项。

网络 Chunk · SSE Event · 模型 Token 不是一回事

常见错误 · 拆包与粘包



✗ 一一对应假设

1 Token = 1 SSE = 1 Chunk

实际：一次 read() 可能只有半条 SSE，也可能包含三条 SSE。
一个模型 delta 可能包含多个 Token，也可能空字符串。

✓ 工程正确姿势

把 Client 写成"无限字节流 + 状态机"

buffer 累积 → lines 切分 → events 解析 → RunEvent。

画三层结构：**Bytes** → **SSE Frames** → **Run Events**。不要画成 Token 与网络包一一对应。

CODE-LEVEL

FastAPI 的 `StreamingResponse` 在 Layer 2/3 之间；浏览器端的 `TextDecoder + atob` 处理 Layer 1/2 之间。Layer 1 的 TCP 内核缓冲由 OS 控制，不会被你写代码改变。

SSE · 基于 HTTP 的服务端单向事件流

WIRE FORMAT

```
id: 17
event: text.delta
data: {"run_id":"run_123","seq":17,"delta":"正在分析"}
```

```
// Heartbeat · 注释行，不进入 SSE 事件语义
: heartbeat
```

```
// Retry · 告诉客户端建议的等待毫秒
retry: 5000
```

```
CONTENT-TYPE: text/event-stream · UTF-8
```

字段含义 · 工程含义

FIELDS

event
事件类型 · Client 用 addEventListener 路由。

data
事件负载 · 通常是 JSON 字符串。

id
事件位置 · 用于 Last-Event-ID 断线重连。

retry
建议重连等待毫秒 · 仅在断线时生效。

空行
一条 SSE 事件的分隔符。

: heartbeat
注释行 · 保持长连接活跃，不应占用 seq。

KEY ENGINEERING RULES

UTF-8 文本协议，毫不能误用 chunked binary 替代。
心跳 ≠ 业务事件，不应占用 Run 的业务 seq。
Content-Type 必须是 text/event-stream ，否则浏览器 EventSource 拒绝连接。

LLM 文本与 Run 状态，优先考虑 SSE

OPTION	DIRECTION	BEST FOR	COST
普通 JSON	请求后一次返回	短请求、完整结果、离线批处理	无增量反馈 · 详见第 11 页下方说明
SSE ✓	服务端 → 客户端	模型输出 · 状态 · 日志流	上行控制(创建/取消)另走普通 HTTP
WebSocket	双向 · 全双工	实时语音 · 多人协作 · 高频双向事件	连接治理复杂 · 必须处理 reconnect / heartbeat

RESTful Split · 三个接口，三个职责

CREATE POST /runs 创建后台任务，立即 202 返回 run_id。	SUBSCRIBE GET /runs/<id>/events SSE 订阅 · 不重新发起模型请求。	CONTROL POST /runs/<id>/cancel 修改控制状态 · Loop 确认后写 run.cancelled。
---	--	---

不要为了一个停止按钮就引入 WebSocket。

创建 / 订阅 / 控制 分离，让 Run 脱离单个浏览器连接继续存在，更容易实现重连。

SSE = 单向事件流 + 简单重连 + 浏览器生态；WebSocket = 双向通道 + 复杂治理，仅在确有需要时引入。

把供应商 Chunk 原样发给前端，会把整个系统绑死

× 反例 · 透明转发

```
# 错误示例
async for chunk in deepseek_stream:
    yield chunk.model_dump_json() + "\n"
```

chunk 来自 OpenAI SDK `ChatCompletionChunk` · Pydantic 模型 · 含 id / object / created / model 等内部字段。

✓ CORRECT BOUNDARY

供应商协议属于 **Model Adapter**；
客户端只依赖 **Harness Event**。



Adapter 处于"协议边界"，Harness Event 处于"业务边界"。

切换 DeepSeek v4-flash → Claude → GPT-5：只替换 Adapter，Client 无需任何修改。

这会导致 4 类系统性后果

- 01

前端依赖具体模型 SDK
dataclass 字段顺序、null 行为、嵌套对象。
- 02

切换模型时所有消费端一起修改
Claude / GPT-5 / DeepSeek 字段差异逐个下沉到前端。
- 03

工具 / 取消 / 重试没有统一事件类型
tool.started 与 run.cancelled 无法共用一套客户端解析器。
- 04

原始对象可能泄露内部字段
prompt 摘要 / system fingerprint 漂到用户侧。

让模型变化停在 Adapter 层

Harness 6 层栈 · 责任对位 · 后续页面都回到这张图



CAPSULE · THE ONE-LINER

Loop 不懂 HTTP · Client 不懂模型 SDK · Gateway 不决定任务是否完成。

用版本化、类型化事件建立稳定协议

EVENT TYPE · 8 + 2 TERMINAL

```
EventType = Literal[
    "run.started",      "text.delta",
    "tool.started",     "tool.completed",
    "run.retrying",     "run.completed",
    "run.failed",       "run.cancelled",
]

class RunEvent(BaseModel):
    schema_version: Literal["1"] = "1"
    run_id: str
    seq: int = Field(ge=0)
    type: EventType
    created_at: datetime
    data: dict[str, Any] = Field(default_factory=dict)
```

Pydantic v2 validate Any publish

关键设计 · 5 项

- ① **schema_version**
强制为 "1"。未知 type 出现时，旧客户端不崩溃。
- ② **run_id + seq**
Run 内的事件位置键值 · 排序 / 去重 / 重放 / Trace。
- ③ **created_at**
客户端可重排 · 可计算 E2E 延迟。
- ④ **EventType Literal**
编译期即可发现拼错 · IDE 自动补全。
- ⑤ **data dict**
schema 可演化字段 · 不破坏旧客户端。

RULE · INTERNAL CONTRACT

run_id + seq = Run 内稳定事件位置

未知事件应被忽略或降级，而不是抛出崩溃。schema_version 是协议演化的护栏。

SSE 编码是独立协议边界，必须可测试

ENCODER · 2 FUNCTIONS

```
def encode_sse(event: RunEvent) → bytes:
    payload = event.model_dump(mode="json")
    lines = [
        f"id: {event.seq}",
        f"event: {event.type}",
        "data: " + json.dumps(payload, ensure_ascii=False),
        "", "",
    ]
    return "\n".join(lines).encode("utf-8")

def encode_heartbeat() → bytes:
    return b": heartbeat\n\n"
```

HEARTBEAT 不得占用业务 seq · 注释行语义

测试矩阵 · 至少 6 项

TC 01	中文 Unicode 往返不丢字
TC 02	换行 在 payload 内部正确转义
TC 03	引号 / 反斜杠 不破坏 SSE 结构
TC 04	空字符串 delta 不漂移 seq
TC 05	大 Payload > 64 KB 单事件
TC 06	拆包 / 粘包 任意字节切分都能恢复

DESIGN PRINCIPLE

把 encode_sse 与 encode_heartbeat 当成纯函数，单测覆盖 6 项矩阵，否则下游所有 bug 都会出现在这里。

直接观察 DeepSeek V4 Flash 原始流

先看清供应商事件，再设计 Adapter · 关闭 SDK 自动重试

```
OPENAI-COMPATIBLE CLIENT

from openai import AsyncOpenAI

client = AsyncOpenAI(
    api_key=os.environ["DEEPSEEK_API_KEY"],
    base_url="https://api.deepseek.com",
    timeout=60.0,
    max_retries=0, # ← 关闭 SDK 重试
)

stream = await client.chat.completions.create(
    model="deepseek-v4-flash",
    messages=messages,
    stream=True,
    max_tokens=1024,
    stream_options={"include_usage": True},
)

max_retries=0 · 由 Harness 接管重试决策
```

为什么关闭 SDK 重试

SDK 默认行为

SDK 会自动重试 2-5 次。L1-L3 错误重试属于"无脑"，失败已经写出去一半就会产生重复事件。

Harness 想要接管

- 何时开始重试 · 在尚未产生业务事件之前
- 重试几次 · 由 emitted_business_event 控制
- 是否可重复 · 由 Run 状态决定
- 是否上报 · 写入 run.retrying

WHY · CLOSE SDK RETRIES

让 Harness 自己决定：何时重试、重试几次、是否已经产生不可重复事件。

文本 / 结束原因 / Usage 出现在不同 Chunk

ADAPTER 不假设任何字段必现 · 4 类必须处理的边界

```
ADAPTER · NORMALIZE

async for chunk in stream:
    choice = chunk.choices[0] if chunk.choices else None

    if choice and choice.delta.content:
        yield {"type": "text.delta",
              "delta": choice.delta.content}

    if choice and choice.finish_reason:
        yield {"type": "model.finished",
              "finish_reason": choice.finish_reason}

    if chunk.usage:
        yield {"type": "model.usage",
              "usage": chunk.usage.model_dump()}

yield → ModelStreamEvent · 不产生 SSE 字符串
```

4 类边界 · Adapter 必须处理

- ⁰¹
Chunk 可能没有 choices
usage-only chunk 出现在末尾。
- ⁰²
delta.content 可能为 None
仅 role 元数据，理应丢弃。
- ⁰³
文本结束 ≠ 业务任务完成
model.finished ≠ run.completed。
- ⁰⁴
Harness 仍需校验 finish_reason
length / content_filter 需要业务层处理。

CHECKLIST · DEEP-SEEK ADAPTER

始终校验 `chunk.choices ≠ None` `choice.delta.content` `finish_reason` `chunk.usage`。

Adapter 产生内部事件，不产生 SSE 字符串

```
PROTOCOL · Adapter

from dataclasses import dataclass
from typing import Protocol, AsyncIterator, Any

@dataclass(frozen=True)
class ModelStreamEvent:
    type: str
    data: dict[str, Any]

class StreamingModel(Protocol):
    async def stream(
        self,
        messages: list[dict[str, str]],
    ) → AsyncIterator[ModelStreamEvent]: ...

任何满足该 Protocol 的类都是合法 Adapter
```

3 种 Adapter · 同一种内部协议

DEEPSEEK

chat.completions

SDK → chunk → normalizer → ModelStreamEvent

RESPONSES API

v1/responses

multi-event → id-merge → ModelStreamEvent

SELF-HOSTED

vLLM / TGI

SSE 字节流 → 同样归一化

↘ ALL → ModelStreamEvent

CP-12 · ADAPTER IS THE BOUNDARY

更换模型，只增加 Adapter；Loop、Event Store 和 Client 不变。

3 个 Adapter · 同一种消费协议 · 不重写下游任何一层。

一个可运行的 Run Store

为什么需要 Run Store · HTTP 连接可能断开，Agent 任务可能仍在执行。

RUN STORE · WHY

任务状态不能依赖单次请求生命周期

- HTTP 连接可能断开，但 Agent 任务可能仍在执行
- 任务状态不能依赖单次请求生命周期
- Run Store 保存任务状态、事件记录和取消信号

支撑 4 项能力

01

断线恢复

02

事件订阅

03

任务取消

04

状态查询

CORE PRINCIPLE

先跑通事件协议，再替换底层基础设施。

先以内存 Run Store 打通事件协议 · 再替换为 PostgreSQL + Redis 通知。

课堂实现

内存保存 Run 状态

- 内存保存 Run 状态
- 维护事件序号
- 保存事件列表
- 提供取消控制

生产环境

PostgreSQL + Redis Stream

- PostgreSQL 保存事实数据
- Redis Stream / PubSub 提供实时通知
- 保证事件可恢复、可追踪

Agent Loop 负责生产事件

Agent Loop 不负责 HTTP 输出，而负责产生业务事实。

LOOP · 6 EVENT TYPES

Loop 应负责产生的事件

task.started
text.delta
tool.started / tool.completed
model.finished / model.usage
run.completed
run.failed / run.cancelled

3 个关键边界

不能混淆的概念

- ① 模型事件 ≠ Run 事件 · 模型输出只是底层数据，需要转换成统一事件协议。
- ② 流结束 ≠ 任务成功 · 正常完成 / 输出截断 / 模型失败 / 用户取消。
- ③ 取消不是普通异常 · CancellableError 是控制信号，需要单独处理。

CORE PRINCIPLE · 5.4

Loop 负责生产事实，HTTP 层负责传输事实。

FastAPI 创建、订阅、取消分离设计

为什么拆成三个接口 · 创建、订阅、控制，三种职责互不重叠。

CREATE · POST /runs

创建任务 ID

- 创建任务 ID
- 启动后台执行
- 立即 202 返回 `run_id`

SUBSCRIBE · GET /runs/{id}/events

SSE 持续推送

- SSE 持续推送
- 支持断线重连
- 不重新发起模型请求

CANCEL · POST /runs/{id}/cancel

停止任务执行

- 停止任务执行
- 通知 Loop 中断
- 写入 `run.cancelled` 终态

SEPARATED · 3 GAINS

分离设计带来 3 项收益

- ✓ 浏览器可以原生使用 EventSource
- ✓ 页面刷新不会重复创建模型调用
- ✓ 客户端断线不会自动取消任务

CORE PRINCIPLE · 5.5

断开连接 ≠ 取消任务。

低延迟转发的常见陷阱

代码写了 `yield` 并不代表链路真的在流式传输。

陷阱 01 · 伪 Streaming

等待完整回答

错误：等待完整回答，一次性返回文本。

问题：用户无法提前看到结果，Streaming 失去意义。

陷阱 02 · 同步 SDK 阻塞

阻塞事件循环

问题：阻塞 Worker，影响其他请求。

解决：优先异步 SDK，或放入受控线程池。

陷阱 03 · 代理缓冲

Nginx / Gateway / CDN

可能重新聚合响应。

检查：关闭代理缓冲 · timeout · 压缩策略。

陷阱 04 · 每个 delta 写数据库

小事件数量巨大 · IO 放大

立即保存 状态事件 · 工具调用 · 审批事件

批量保存 文本 delta

陷阱 05 · 断连直接取消

用户刷新 → Agent 被取消？

区分：订阅断开 ≠ Run 取消

· 用户主动停止：取消任务

· 临时断线：继续执行

· 长时间无人订阅：后台清理

DEBUG ORDER · 全链路定位

模型首事件 → Adapter → Event Store → Gateway flush → Proxy → Browser render

EventSource：创建与订阅分离

浏览器消费 Streaming · POST 创建 → GET 订阅 → 按 seq 顺序消费。



CORE PRINCIPLE · 6.1

前后端职责边界清晰 · 客户端只渲染事件，不解释业务。

为什么需要渲染节流

每个 delta 都更新页面 · 重复 Markdown 解析 · 高频 DOM 更新 → 浏览器性能下降。

PROBLEM

每个 delta 引起的副作用

- 浏览器性能下降
- 重复 Markdown 解析
- 高频 DOM 更新

SOLUTION · 分两层处理

网络高频收，UI 合并画

- 网络层**：高频接收 delta
- UI 层**：合并刷新 (rAF / debounce)

CLIENT CODE · RAF

```
let scheduled = false;
function scheduleRender() {
  if (scheduled) return;
  scheduled = true;
  requestAnimationFrame(() => { output.textContent = accumulatedText; scheduled = false; });
}
```

CORE PRINCIPLE · 6.2

网络可以高频收，界面不必高频画。

fetch + ReadableStream

EventSource 不够用时：POST Streaming · Authorization Header · 更灵活控制。

适用场景

什么时候放弃 EventSource

- 需要 POST Streaming
- 需要 Authorization Header
- 更灵活控制请求 body / 重试

关键处理

4 个字节流陷阱

- × 字节流解码 · UTF-8 边界
- × SSE 帧切分 · 不是所有 read() 是一条
- × 多事件处理 · 一个 chunk 可能 3 条
- × buffer 必须自己拼接

CLIENT · BUFFER PATTERN

```
const reader = response.body!.getReader();
const decoder = new TextDecoder("utf-8");
let buffer = "";
while(true) {
  const { value, done } = await reader.read();
  if (done) break; buffer += decoder.decode(value, {stream:true});
}
```

CORE PRINCIPLE · 6.3

chunk ≠ event · event ≠ token · 必须通过 buffer 解析完整事件。

Python CLI 消费 SSE

Web、CLI、IDE 共享同一个 Harness Event 协议。

CLI · ONLY

CLI 只负责

- 接收事件
- 去重
- 展示结果

CLI · CODE

```
async with httpx.AsyncClient(  
    timeout=None  
) as client:  
    async with client.stream(  
        "GET", events_url  
    ) as r:  
        async for line  
            in r.aiter_lines():  
  
            print  
                data,  
                end="",  
                flush=True)
```

CLI · NOT

CLI 不负责

- × 模型调用
- × 工具执行
- × 重试逻辑

ONE PROTOCOL · 3 CLIENTS

3 套客户端 · 1 套 Harness Event

WEB

EventSource · 视觉富文本

CLI

ANSI 颜色 + 流式打印

IDE

Webview 面板

CORE PRINCIPLE · 6.4

客户端差异只停留在渲染层 / IO 层。

客户端取消

AbortController 的限制 · 只停止当前 HTTP 请求，不能保证后台 Agent 停止。

LIMITATION

AbortController 只能做

- ✗ 当前 HTTP 请求
- ✗ 不能保证后台 Agent 停止
- ✗ 不能保证模型停止计费

CORRECT FLOW · 3 步

正确的客户端取消

- ① 停止本地读取
- ② 调用 Cancel API (`POST /runs/{id}/cancel`)
- ③ 服务端传播取消信号 → Loop / Model / Tool

CLIENT CODE

```
const controller = new AbortController();
eventSource.addEventListener("text.delta", render);

async function cancelRun(runId) {
  controller.abort(); // stop local read
  await fetch(`/runs/${runId}/cancel`, {method:"POST"});
}
```

CORE PRINCIPLE · 6.5

AbortController 只能停"看"。真正"停做"必须走 Cancel API。

关闭连接 ≠ 停止生成

Gateway 已断开 · Agent Loop / 模型 / 工具 / 外部副作用仍在持续发生。

KEEPS RUNNING · 仍会发生

关闭订阅后 仍会发生

- × Gateway 已断开
- × Agent Loop 继续执行
- × 模型仍生成
- × 工具仍运行
- × 外部副作用继续发生

DISTINCTION

断开的是订阅连接，
停止的是业务 Run。

- `eventSource.close()` = 客户端不再读
- Run 状态由服务端独立维护
- 取消须走 explicit API

DIAGRAM · 订阅 vs 业务

Subscription → GET `/runs/{id}/events` (`close()` 即可停)
Run state → POST `/runs/{id}/cancel` (需 `explicit`)

Cooperative Cancellation 与强制终止

两种取消方式 · 协作式取消 vs 强制 Task Cancel。

COOPERATIVE · 协作式取消

组件主动检查取消信号

优点 · 安全 · 可保存状态 · 可释放资源

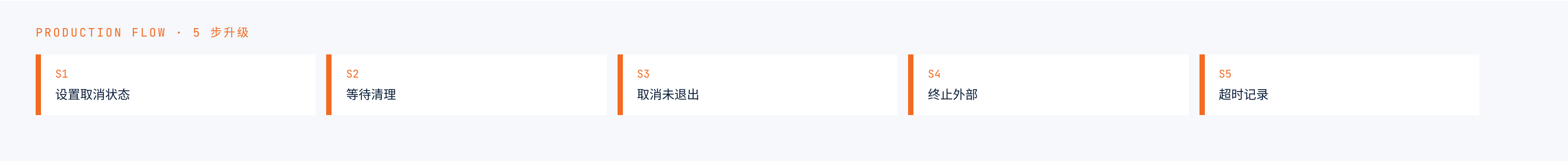
流程 · 每步 await 之前检查 token

FORCED · 强制 Task Cancel

Task.cancel() 解除阻塞

适合 · 长时间阻塞任务

代价 · 中断点不确定 · 副作用未保存



CORE PRINCIPLE · 7.2

不要用 `except Exception` 吞掉 `CancelledError`。

工具取消比模型取消更复杂

不同副作用的工具 · 必须分别建模取消语义。

TYPE	示 例	取 消 策 略
可重试 只读	搜索 · 读取文件	停止等待 · 无副作用可重复
中等代价 长任务	Shell · 测试 · 构建	保存进程句柄 · 发送 SIGTERM
不可逆 副作用	邮件 · 支付 · 发布	查询外部状态 · 禁止盲目重放

KEY MECHANISM · 4 项

①
幂等 Key

②
Tool Call ID

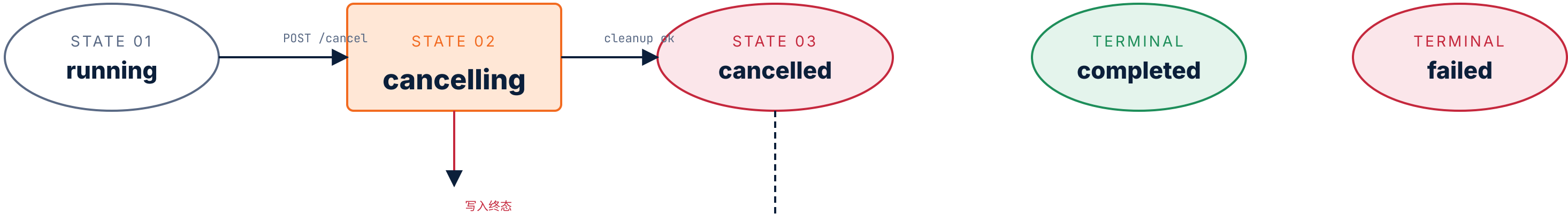
③
状态查询

④
人工确认

超时说明"未收到结果"，不代表"动作未发生"。**unknown** 是必须被建模的状态。

取消状态机

取消不能直接 running → cancelled，必须经过 cancelling。



REASON · 为什么不在直接取消

需要 cancelling 过渡态

- 资源释放需要时间
- 避免完成与取消竞争
- 服务端可显式记录"还在取消中"

SAFE SQL · CAS

```
UPDATE agent_runs
SET status = 'cancelling'
WHERE run_id = :run_id
AND status IN ('created', 'running');
```

取消后的部分输出处理

取消后必须保存 · 已输出文本、最后 seq、Loop 步骤、工具状态、Token 使用、取消原因。

CANCEL SAVE · 6 FIELDS

取消时必须保存的内容

- 已输出文本
- 最后事件序号 (last_seq)
- 当前 Loop 步骤
- 工具状态 (call_id · 是否已发出)
- Token 使用量
- 取消原因 (cancelled_at)

NOTE · 检查点 ≠ 续跑点 · 部分 JSON / SQL / Patch 不能直接提交。

HALF-FINISHED · 不可提交

半成品不能直接提交

- × 不完整的 JSON
- × 半执行的 SQL
- × 拼接中断的 Patch

REPLAYABLE · 可被客户端重放

仍可被 replay

- ✓ 最后一段文本
- ✓ 终态 run.cancelled
- ✓ 已用 Token · 已跑 Tool

CORE PRINCIPLE · 7.5

取消即存档点 · 不等于可执行点。

Streaming 失败阶段

越晚失败 · 已产生事实越多 · 越不能直接重试。

一次 Streaming 的 5 个阶段



DECISION CHAIN · 5 阶段

不同阶段对应不同策略

A · 连接 / 响应 · DNS · 429 · 503 → 有限重试

B · 首事件后 · 网络断 → **禁止**盲目拼接重跑

C · 大量事件 · 必须先查询上次状态，再决定重放还是重跑

为什么不能简单重试模型

第一次输出到一半 · 重新请求 · 可能生成另一条路径 → 拼接语义冲突。

第 1 次 · 输出到一半

「第一，修改缓存键；第二，监控命中率...」

重新请求

「先增加数据库索引，缓存层改异步...」

语义冲突 · 状态错误

IF 直接拼接

两条互相打架的建议

MUST · 区分 3 种恢复

Transport Resume / Model Retry / Task Resume — 三者不是同一个概念。

R-01

Transport Resume

恢复事件传输。
走 Event Store 重放。

R-02

Model Retry

重新调用模型，
仅当尚未产生业务事件。

R-03

Task Resume

从 Checkpoint
继续下一步。

有边界的连接重试

只有尚未产生不可重复业务事件时才能自动重试。

ALLOW · 白名单

允许重试

✓ DNS 错误

✓ 网络失败

✓ 429

✓ 503

DENY · 黑名单

不能机械重试

✕ 认证失败

✕ 参数错误

✕ Schema 错误

✕ 内容拒答

LIMITS · 6 项必限制

max_attempts=N · max_total_seconds=T · whitelist={429·5xx·DNS} · exponential backoff · jitter · Run budget

CRITICAL CHECK

是否已发出业务事件？

未发出：有限重试

已发出：走 Replay · 重新调用 model 将产生第二份独立输出

断线续传核心：事件重放

断线续传恢复的是 Harness Event，不是模型 Token。

CORE CONCLUSION

断线续传恢复的是 Harness Event **不是** 模型 Token。

CLIENT · 职责

客户端保存

- Last-Event-ID
- 重连时通过 header 回传

SERVER · 职责

服务端按 seq 查询

- `SELECT * ... seq > last`
- 重放顺序 = 写入顺序

RESULT · 不重跑

效果

- ✓ 不重新调用模型
- ✓ 不产生重复任务
- ✓ 客户端按 seq 去重

DISTINCTION

"恢复连接 ≠ 重新调用模型"

断线续传最小单元是 Harness Event 而非 Token · 客户端永不因网络抖动被重复计费。

Event Store 与实时通知

职责分离 · PostgreSQL 保事实 · Redis 做通知。

FACTS · PostgreSQL

保存事实事件

- 强一致 · 可重放
- 唯一约束按 (run_id, seq)
- 永远不能丢事实

NOTIFY · Redis

实时流通知

- 不重 · 可丢 · 高速
- 客户端按 seq 去重
- 不可作为事实来源

FLOW · Loop → Store → Notify → Gateway → Client

(详见下方)



CORE PRINCIPLE · 8.5

通知可以丢 · 事实事件不能丢。

长文本为什么需要 Checkpoint

Streaming 解决"看得快", 但解决不了"Context Window 限制"。

STREAMING 解决不了

已知能力的边界

- × Context Window 限制
- × 一次性读不完所有 token
- × Worker 中途崩溃 = 任务丢失

LONG TASK · 需要

长任务的 3 项配套

- ✓ 分步骤执行
- ✓ 保存中间状态
- ✓ 支持恢复

CHECKPOINT · 6 字段

① 当前步骤

loop_step

② 已完成内容

completed[]

③ 下一步位置

next_cursor

④ 上下文摘要

context_digest

⑤ 工具状态

tool_state[]

⑥ checkpoint 版本

version

CORE PRINCIPLE · 8.6

Checkpoint 保存可恢复状态 · 不是把全部历史塞回 Prompt。

会话断开与进程崩溃

两类问题 · Recover from 客户端断线 vs Worker 崩溃。

客户端断线

Browser subscribe lost

恢复：

重新订阅 (Last-Event-ID)

重放事件

模型继续工作

Worker 崩溃

Process died · Work in memory lost

恢复：

读 checkpoint

重启 worker → 从 Checkpoint 继续执行

已落库事实保留

RECOVERY MATRIX · 4 CONDITIONS			
① 客户端断线 + 短暂 → 重新订阅 → 服务端无需任何动作		② 客户端断线 + 长期 → 服务端后台清理 Run → 通知前端已回收	
③ Worker 崩溃 + 未落地 CP → 读 Checkpoint 恢复 → 重新跑未完成的步骤		④ Worker 崩溃 + 涉及外部副作用 → unknown 状态 → 必要时人工确认	

错误事件协议

客户端需要稳定错误码 · 服务端保存事实 · 不暴露敏感信息。

CLIENT NEEDS · 4 项

客户端能看到

- 稳定错误码 `STABLE_CODE`
- 是否可重试
- 恢复方式
- 失败阶段定位

SERVER KEEPS · 5 项 + 3 防泄漏

- 原始异常
- Trace ID
- Request ID
- Token 使用
- 工具副作用状态

DO NOT 暴露：

- × API Key
- × Prompt 原文
- × 内部堆栈

EXAMPLE · `run.failed` EVENT

```
{ "type": "run.failed",
  "data": {
    "code": "TOOL_TIMEOUT",
    "stage": "tool.completed",
    "retryable": false,
    "hint": "需要确认是否属于 unknown 状态"
  }
}
```

Loop 不应该直接操作 HTTP

Agent Loop 生产事件 · SSE Gateway 传输事件 · 责任分离。

错误 · WRONG

Agent Loop 同时

×

调模型

×

拼 SSE 字符串

×

管理 HTTP 连接

正确 · RIGHT

Agent Loop 只

✓

驱动任务

✓

调工具

✓

产生事件

SSE Gateway 只

SSE Gateway 负责

✓

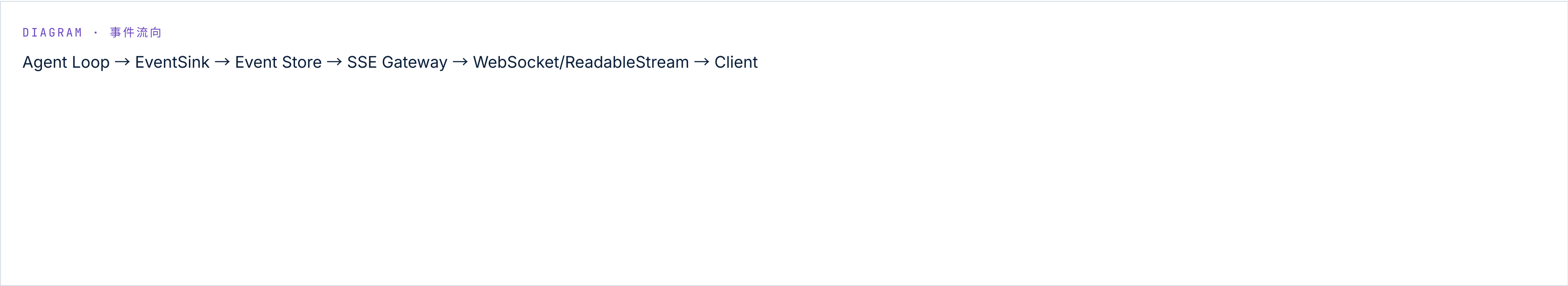
编码

✓

心跳

✓

订阅管理



Harness 各层职责

6 层责任对位 · 跨层耦合的 bug 比想象中更常出现。

组 件	负 责	不 负 责
Model Adapter	模型流转换	UI 输出
Agent Loop	执行任务	HTTP
Tool Runtime	工具执行	页面展示
Event Store	保存事件	重新执行
SSE Gateway	推送事件	业务决策
Client	展示和取消	模型解析

CP - 9.2

每个组件只关心自己的"应做"与"不应做"。

Trace 必须记录

至少包含 · Trace 是设计的一部分 · 不是出问题才查。

TRAIL · 14 fields

trace_id · run_id · 模型版本 · Prompt 版本

首 Token 时间 · 总耗时 · Token 使用

重试次数 · 重连次数 · 取消延迟 · 工具调用 · checkpoint

最终状态 · 错误码

WHY · TRACE 设计为第一性

Trace 不是运维工具 · 是 Harness 的设计输入。

测试重点

不要只测"页面会不会打字" · 11 项边界必须覆盖。

DON'T · 不应只测

"页面是否逐字出现"

- 一只 index.html
- 一只 EventSource
- 没有边界测试 = 上线 surprise

MUST · 必须覆盖 11 项

必须覆盖的边界

- SSE 编码 · 拆包恢复 · 粘包处理
- seq 顺序 · 事件去重
- 断线续传 · 取消传播
- 完成 / 取消 竞态 · 流失败恢复 · checkpoint 恢复

EXAMPLE TEST · 拆包恢复

```
def test_sse_chunk_split():
    data = "id: 17\nevent: text.delta\nda|ta: {\\"seq\\":17}"
    events = list(parser.feed_bytes(
        data.encode("utf-8")
    ))
    assert events[0].seq == 17
```

CP-9.4

Harness 的 bug 都是"小颗粒度边界条件"，不是大架构错误。

课后练习：Streaming Agent Gateway

将同步模型调用升级为 · 可观察 · 可取消 · 可恢复 · 可审计。

目标 · 4 UPGRADES

4 项能力升级

✓ 可观察 · 事件可追踪

✓ 可取消 · 显式终止

✓ 可恢复 · Replay + Checkpoint

✓ 可审计 · Trace + 失败协议

8 项系统能力

学员完成时应能实现

· 创建独立 Run

· 流式调用模型

· 转换统一 Harness Event

· SSE 推送 · 用户取消 · 断线恢复

· checkpoint 保存 · TTFT 统计

IMPLEMENT ORDER · 11 步

1 · 定义 RunEvent

2 · 实现 SSE 编解码

3 · 抽象 StreamingModel

4 · 接入模型 Adapter

5 · 实现 Run Store

6 · 实现 Agent Loop

7 · 创建订阅取消接口

8 · 开发 Web/CLI 客户端

9 · 增加异常测试

10 · 接入数据库与 checkpoint

ACCEPTANCE · 10 标准

首个 delta 快速展示 · 不依赖模型 SDK · Run 单一终态 · 取消真停止 · 断线不重跑

本节小结

"Streaming 不是让模型更快，而是让 Agent 执行过程可见、可停、可续。"

8 CORE CONCLUSIONS

八个核心结论

- 01 · Streaming 优化感知延迟 · 不降低模型推理时间
- 02 · SSE 是传输协议 · Harness Event 是业务协议
- 03 · Model Adapter 屏蔽供应商差异
- 04 · 客户端需处理拆包 · 顺序 · 重复 · 渲染压力
- 05 · 取消必须贯穿模型 · 工具 · 状态机
- 06 · 已产生业务事件后不能盲目重新生成
- 07 · 断线恢复依赖 Event Store · 任务恢复依赖 Checkpoint
- 08 · Streaming 终极目标 · 让 Agent Loop 可观察 · 可中断 · 可恢复 · 可审计

CAPSULE

让 Agent Loop 可观察 · 可中断 · 可恢复 · 可审计。